

UNIVERSIDAD AMERICANA



Programación Orientada a Objetos

❖ Desarrollo de una API con Spring Boot

Elaborado por:

Jonathan Rivera Guido

Docente: José Durán García

Fecha de entrega:

13/05/2026

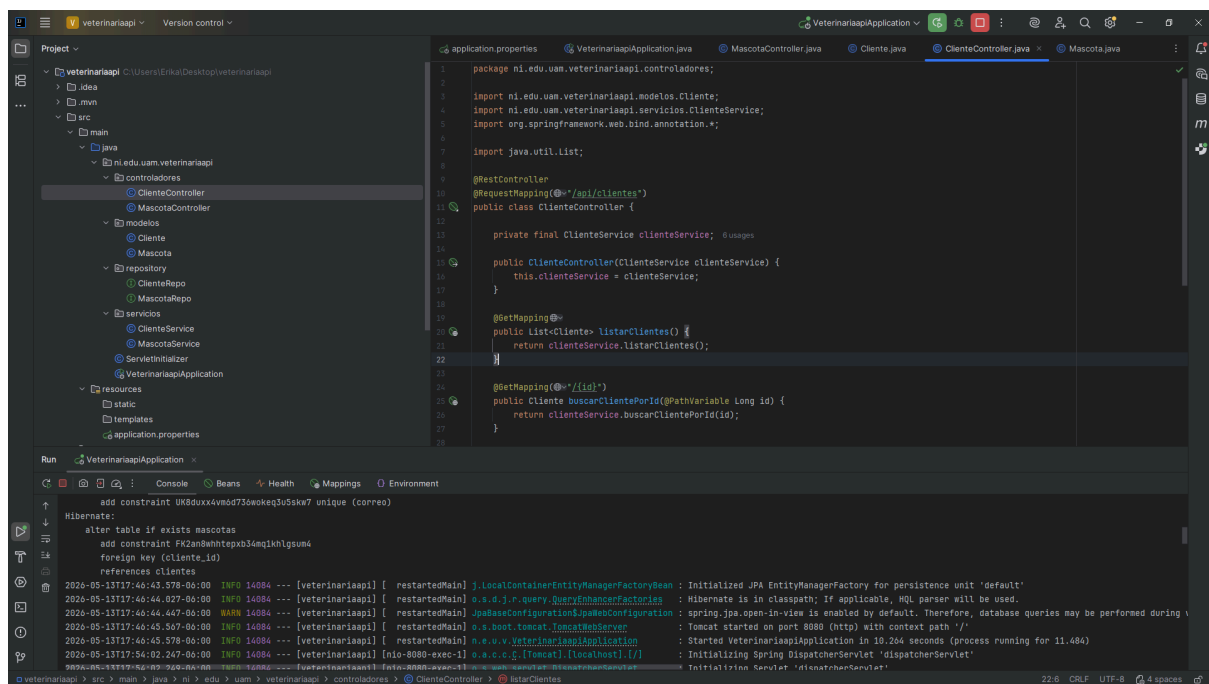
Nombre de programa: Veterinariaapi

- Descripción del proyecto

El presente proyecto consiste en el desarrollo de una API REST elaborada con Spring Boot y Java, orientada a la gestión básica de una veterinaria. La aplicación permite administrar información de clientes y mascotas mediante operaciones CRUD, es decir, crear, consultar, actualizar y eliminar registros. Para cumplir con la relación entre clases, se implementó una asociación entre las entidades Cliente y Mascota, donde un cliente puede tener una o varias mascotas registradas, mientras que cada mascota pertenece a un cliente específico.

La API utiliza JPA/Hibernate para manejar la persistencia de datos y PostgreSQL como base de datos relacional. Además, los endpoints fueron probados mediante Postman, verificando que las operaciones funcionen correctamente y que la relación entre las entidades se refleje en las respuestas del sistema. De esta manera, el proyecto demuestra el uso de una arquitectura básica por capas, compuesta por modelos, repositorios, servicios y controladores.

1. IntelliJ ejecutando el proyecto



2. Creación de cliente

Get:

The screenshot shows the Postman interface with a workspace named "JONATHAN JOSUE RIVERA GUIDO's Workspace". On the left, a collection named "My Collection" contains several requests, with "GET Get data" selected. The main panel displays the details of this request, which is a GET method to the URL "http://localhost:8080/api/clientes". The "Body" tab is active, showing a "none" body type. Below the request details, the response is shown as a 200 OK status with a response time of 54 ms and a size of 268 B. The response body is a JSON array with one object, which is displayed in a formatted view:

```
1 [
2   {
3     "id": 3,
4     "nombre": "Jonathan",
5     "apellido": "Rivera",
6     "correo": "jonathan.rivera@uam.edu.ni",
7     "telefono": "8444-0122"
8   }
9 ]
```

Post:

The screenshot shows the Postman interface with the same workspace. The selected request is "POST Post data" from the "My Collection". The request details show a POST method to the URL "http://localhost:8080/api/clientes". The "Body" tab is active, showing a JSON body type with a raw view of the request body:

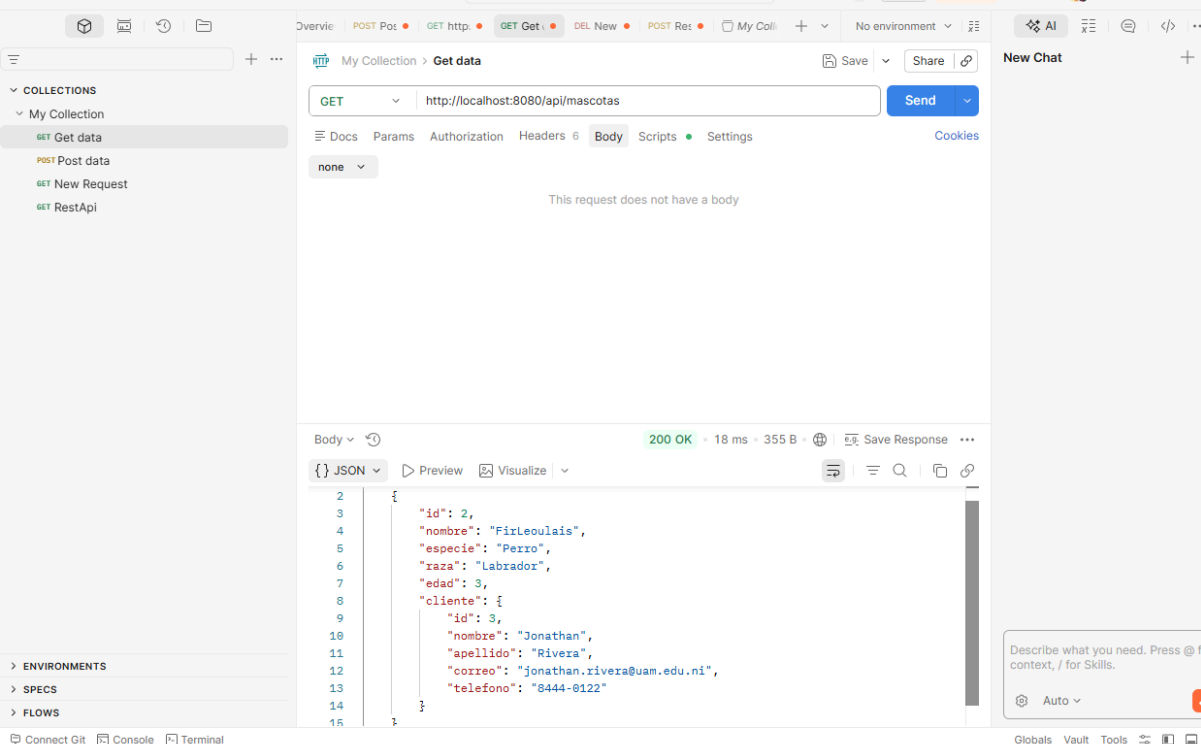
```
1 {
2   "nombre": "Jonathan",
3   "apellido": "Rivera",
4   "correo": "jonathan.rivera@uam.edu.ni",
5   "telefono": "8444-0122"
6 }
```

Below the request details, the response is shown as a 200 OK status with a response time of 64 ms and a size of 266 B. The response body is a JSON object, displayed in a formatted view:

```
1 {
2   "id": 3,
3   "nombre": "Jonathan",
4   "apellido": "Rivera",
5   "correo": "jonathan.rivera@uam.edu.ni",
6   "telefono": "8444-0122"
7 }
```

3. Creación de mascotas:

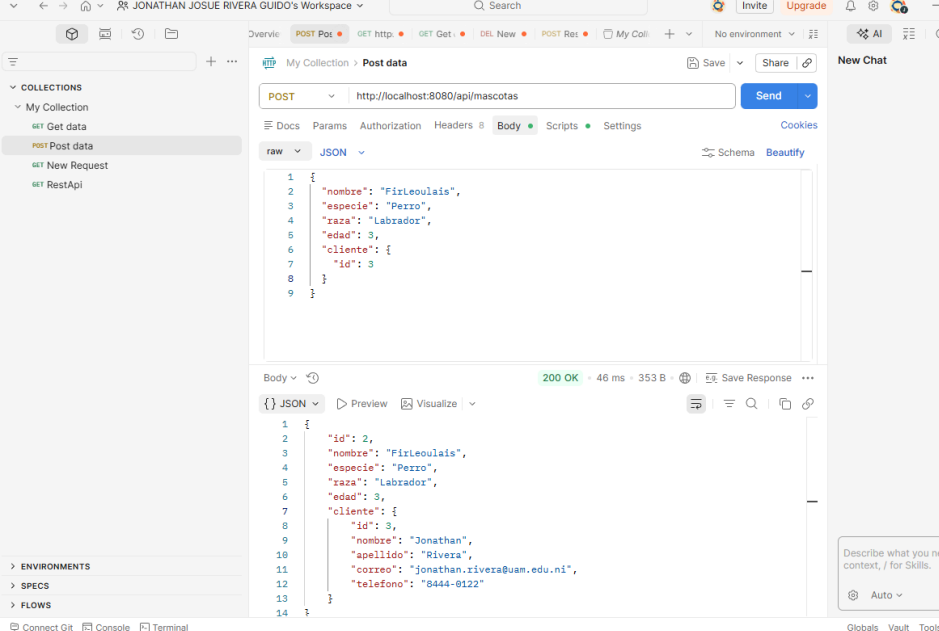
Get:



The screenshot shows the Postman interface with a GET request to `http://localhost:8080/api/mascotas`. The response is a 200 OK status with a 18 ms response time and 355 B of data. The response body is a JSON object:

```
{
  "id": 2,
  "nombre": "Firleoulais",
  "especie": "Perro",
  "raza": "Labrador",
  "edad": 3,
  "cliente": {
    "id": 3,
    "nombre": "Jonathan",
    "apellido": "Rivera",
    "correo": "jonathan.rivera@uam.edu.ni",
    "telefono": "8444-0122"
  }
}
```

Post:



The screenshot shows the Postman interface with a POST request to `http://localhost:8080/api/mascotas`. The request body is a JSON object:

```
{
  "nombre": "Firleoulais",
  "especie": "Perro",
  "raza": "Labrador",
  "edad": 3,
  "cliente": {
    "id": 3
  }
}
```

The response is a 200 OK status with a 46 ms response time and 353 B of data. The response body is a JSON object:

```
{
  "id": 2,
  "nombre": "Firleoulais",
  "especie": "Perro",
  "raza": "Labrador",
  "edad": 3,
  "cliente": {
    "id": 3,
    "nombre": "Jonathan",
    "apellido": "Rivera",
    "correo": "jonathan.rivera@uam.edu.ni",
    "telefono": "8444-0122"
  }
}
```

Put:

The screenshot displays the REST Client interface. On the left, a sidebar shows a collection named 'My Collection' with several requests, including 'New Request' which is currently selected. The main workspace is titled 'New Request' and shows a PUT request to the URL 'http://localhost:8080/api/mascotas/2'. The request body is a JSON object:

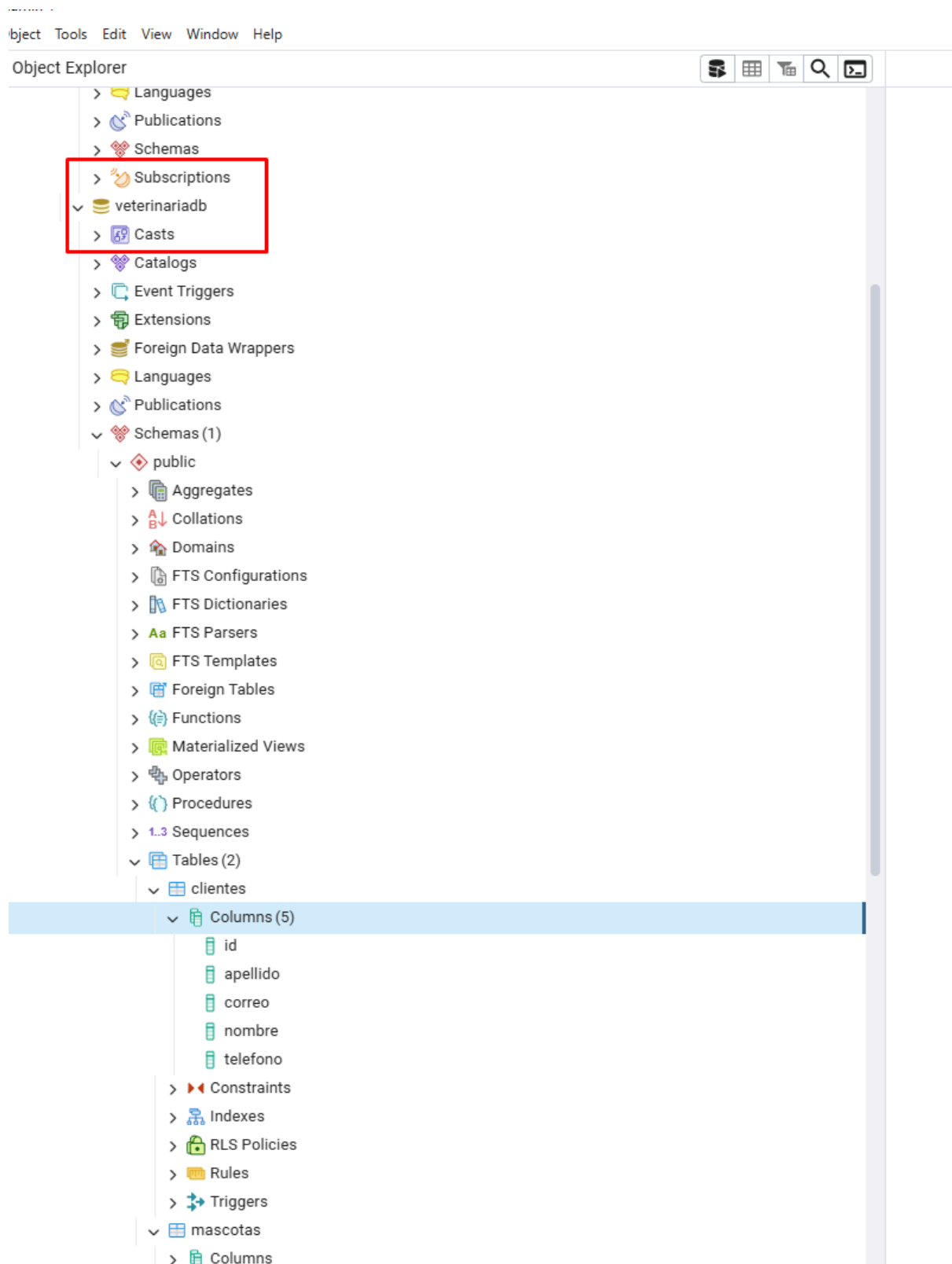
```
{  "nombre": "Leo",  "especie": "Perro",  "raza": "Labrador",  "edad": 3,  "cliente": {    "id": 3  }}
```

. Below the request, the response is shown as a 200 OK status with a response time of 55 ms and a body size of 345 B. The response body is a JSON object:

```
{  "id": 2,  "nombre": "Leo",  "especie": "Perro",  "raza": "Labrador",  "edad": 3,  "cliente": {    "id": 3,    "nombre": "Jonathan",    "apellido": "Rivera",    "correo": "jonathan.rivera@uam.edu.ni",    "telefono": "8444-0122"  }}
```

. The bottom of the interface includes a status bar with 'Connect Git', 'Console', and 'Terminal' buttons, and a right sidebar with 'New Chat' and an 'Auto' button.

BASE DE DATOS EN PG4 ADMIN:



Link de repositório: